


[Products and Technology](#)
[Groups](#)
[Developer s](#)
[Partners](#)
[Events](#)
[Help Center](#)
[Topic Pages](#)
[Explore SAP](#)

[SAP Community](#) / [Developers](#) / [Artificial Intelligence](#) / [Blogs Posts](#) / [Agentic Harness Architecture: Seven Pillars That M...](#)

Artificial Intelligence Blogs Posts

AI Search



Search

Agentic Harness Architecture: Seven Pillars That Make Claude Code Production-Grade


SAP Mohan-Sharma

Product and Topic Expert

[Sign In to Subscribe](#)


2026 May 13 1:27 PM



10 Likes



1,122

SAP MANAGED TAGS

Artificial Intelligence

The Agentic Harness Architecture: Making Claude Code Production-Grade

Claude Code ships with primitives - skills, memory, subagents, MCP servers, rules, hooks, and settings. Out of the box, they are individual features. Composed deliberately, they become a production harness - a meta-architecture that makes your AI tool predictable, context-efficient, and capable of improving with every use.

This post teaches the harness architecture itself. Not one specific implementation, but the composable system that turns Claude Code's primitives into a production-grade tool. I built this for a cross-system enterprise investigation workflow, but the architecture applies to any domain where you need reliable, repeatable, shareable AI workflows.

I will be explicit about what Claude Code gives you for free versus what you architect on top.

What Is a Harness?

A harness is the architectural layer between "raw AI tool" and "production system." Claude Code provides powerful primitives. A harness composes those primitives - and adds custom patterns on top - to answer seven questions:

1. How do you keep context lean across multi-turn conversations? (**Context Engineering**)
2. How does the system learn and improve over time? (**Memory Management**)
3. How do you encode repeatable workflows? (**Skills**)
4. How do you orchestrate multiple specialized agents? (**Agent Contracts**)
5. How do you avoid expensive work when cheap work suffices? (**Tiered Execution**)
6. How do you keep computation reliable? (**Deterministic Engines**)
7. How do you share it with your team? (**Portability**)

Some answers come directly from Claude Code's built-in features. Others require custom architecture on top. The harness is knowing which is which and composing them into a system greater than the sum of its parts.

The Primitives: What Claude Code Gives You

Before building a harness, understand what you get for free:

Primitive	What It Does	Location
CLAUDE.md	Instructions always in context, every turn	./CLAUDE.md , ./ .claude/CLAUDE.md
CLAUDE.local.md	Local instructions (not committed to git)	./CLAUDE.local.md
Auto Memory	Claude's notes, MEMORY.md index loads at startup	~/.claude/projects/{project}/memory/
Rules	Modular instructions, optionally path-scoped	.claude/rules/*.md
Skills	Workflow files with lazy-loaded content	.claude/skills/name/SKILL.md
Subagents	Isolated context windows, custom definitions	.claude/agents/*.md + Agent tool
MCP Servers	External tool integration via JSON config	.mcp.json
Settings	Permissions, env vars, model config	.claude/settings.json
Hooks	Lifecycle event handlers (shell commands)	Settings files, hooks events
Auto-compaction	Automatic summarization when context fills	Built-in, triggers automatically

These are the building blocks. Now here is what you architect on top.

Pillar 1: Context Engineering

This is the most important pillar and the one that requires the most custom architecture. Claude Code gives you auto-compaction (it summarizes when context fills) and subagent isolation (Agent tool creates fresh context windows). Everything else is your responsibility.

What Claude Code Provides (Built-in)

- **Auto-compaction.** When context approaches the limit, Claude Code clears older tool outputs first, then summarizes the conversation if needed. CLAUDE.md is re-read from disk and re-injected after compaction. Invoked skills are re-attached (first 5,000 tokens each, combined budget of 25,000 tokens, most recent skills prioritized).
- **Subagent context isolation.** The Agent tool spawns a subagent with its own fresh context window. Only the final result returns to the parent. Heavy data in the subagent is garbage collected when it completes.
- **Skill progressive disclosure.** Only the skill's name and description from frontmatter load at startup (~100 tokens per skill). The full SKILL.md body loads only when invoked. Supporting files (scripts, references) load only when Claude reads them.
- **Memory auto-load.** First 200 lines or 25KB of MEMORY.md (whichever comes first) loads at session start. Topic files load on demand when Claude needs them.

What You Build (Custom Architecture)

- **Token budget discipline.** Claude Code does not budget tokens for you. You must decide: what goes in CLAUDE.md (always in context), what goes in rules (conditionally loaded), what goes in skills (lazy-loaded), and what stays on disk (never in context unless explicitly read). This is a design decision, not a feature.
- **File-on-disk boundaries.** This is a custom architectural pattern, not a Claude Code feature. You write data to files (e.g., .claude/cache/{id}/) using Python scripts, then have subagents read specific files via the Read tool. This keeps heavy data out of the main context entirely - it is never loaded unless an agent explicitly reads it.
- **Structured summaries across boundaries.** When a subagent completes, it returns text. You design what that text contains - typically a structured JSON summary of ~500 tokens rather than raw data. The Agent tool does not enforce structure; your agent prompt does.

The Token Budget (Your Design)

Startup (Claude Code loads these automatically):

What	Size	Loaded By
CLAUDE.md	~1,500 tokens	Built-in (always in context)
Skills metadata (name + description only)	~500 tokens	Built-in (progressive disclosure)
Memory (first 200 lines or 25KB of MEMORY.md)	~500 tokens	Built-in (auto-load)
AUTOMATIC TOTAL	~2,500 tokens	

Per-turn (your architecture controls these):

What	Size	Controlled By
------	------	---------------

Active skill body (loaded on invocation)	~4,000 tokens	Your skill design
Routing result (lightweight MCP call)	~100 tokens	Your tier logic
Subagent result (structured summary)	~500 tokens	Your agent contract
User-facing explanation	~1,000 tokens	Your skill steps
PER-TURN ADDITIONS	~5,600 tokens	

What never enters main context (your architecture):

Data Type	Size	Where It Lives
Raw API responses	20K-50K	On disk, read by subagent only
Integration logs	50K+	On disk, read by subagent only
Comparison data	10K-20K	On disk, read by subagent only

These exist only in subagent contexts that are destroyed after use. The discipline is yours. Claude Code gives you the primitives (subagents, lazy skills, auto-compaction). You decide what goes where.

Pillar 2: Memory Management

What Claude Code Provides (Built-in)

- **Auto memory.** Claude writes notes to `~/.claude/projects/{project}/memory/`. `MEMORY.md` acts as an index (first 200 lines or 25KB loads at session start). Topic-specific files are loaded on demand when Claude needs them. Per-repository, shared across worktrees.
- **Rules directory.** `.claude/rules/*.md` files. Rules without a `paths:` frontmatter field load unconditionally at launch (always in context). Rules with `paths:` load only when Claude works with matching files.
- **CLAUDE.md.** Always in context. Version-controlled. Your most important instructions go here.

What You Build (Custom Architecture)

- **Three-bank categorization.** Claude Code's auto-memory is flat. You impose structure by organizing memories into categories: facts (permanent system knowledge), episodes (time-bound investigation outcomes), and procedures (learned strategies). This categorization is your convention, not a Claude Code feature.
- **Memory lifecycle.** Retrieval-before-every-turn is not automatic for rules or memory topics beyond `MEMORY.md`'s first 200 lines. You design the retrieval strategy in your skill's workflow steps.
- **Promotion workflow.** Moving validated patterns from personal memory to shared `.claude/rules/` is a team practice you establish, not a product feature.

The Two-Tier Model (Your Architecture)

Tier 1: Shared Knowledge - `.claude/rules/`, committed to git

File	Purpose
error-patterns.md	Confirmed error codes and their meaning
remediation-playbook.md	Known fixes per error type
team-contacts.md	Escalation paths per system
system-knowledge.md	API quirks, rate limits, known behaviors
grounding-rules.md	Anti-hallucination constraints

Everyone gets this on `git pull`. New team member = 90% value on day one.

Tier 2: Personal Memory - `~/.claude/.../memory/`, per-user

Claude auto-writes investigation outcomes. Experienced users accumulate pattern recognition over time. New users start empty but Tier 1 carries them.

Promotion workflow (your team practice):

1. User discovers pattern - Claude saves to personal memory
2. User validates it is real (not one-off)

3. User adds to `.claude/rules/` and runs `git push`
4. Entire team benefits immediately

Pillar 3: Skills as Workflow Encoding

What Claude Code Provides (Built-in)

- **SKILL.md format.** YAML frontmatter (`name`, `description`, `when_to_use`, `allowed-tools`, `context`, `agent`, `model`) plus markdown body. Official feature with progressive disclosure - only the description loads at startup.
- **Automatic invocation.** Claude matches user intent against skill descriptions and invokes automatically. Users can also invoke directly via `/skill-name`. Use `disable-model-invocation: true` for skills only you should trigger.
- **Context modes.** Set `context: fork` to run in a subagent. Set `agent: Explore` (or any custom agent) to pick the execution environment. Default runs inline in main context.
- **Post-compaction survival.** After auto-compaction, the most recent invocation of each skill is re-attached (first 5,000 tokens each). Skills share a combined 25,000-token re-attachment budget. Most recently invoked skills are prioritized - older skills may be dropped if you invoked many in one session.
- **Supporting files.** Skills are directories, not single files. A skill can bundle scripts, templates, reference docs, and examples alongside SKILL.md.

What You Build (Custom Architecture)

- **Multi-step deterministic workflows.** A skill body that orchestrates: routing API call, Python script execution, subagent spawning, verification script, report generation, and memory write. The SKILL.md format and progressive disclosure are official; the workflow design is yours.
- **Bundled scripts and tools.** The skill directory can contain Python scripts that Claude executes. Your `tools/fetch_data.py`, `tools/compare_data.py`, and `tools/generate_report.py` live alongside SKILL.md. Claude runs them via Bash - the script code never enters context, only its output.
- **Error handling and degradation.** Skills defining what happens when a system is unreachable, when a script fails, when an agent produces low-confidence results. These are your design choices encoded in the skill body's constraint section.

Skill Anatomy

File: `.claude/skills/investigate/SKILL.md`

```

1  ---
2  name: investigate
3  description: Investigate a record across multiple backend systems
4  when_to_use: Use when user says "investigate", "trace", "find",
5    "check status", or "why didn't this sync"
6  allowed-tools: Bash(python3 tools/*) Read
7  context: fork
8  agent: general-purpose
9  ---
10
11 ## Steps
12 1. Call routing MCP tool to determine investigation path
13 2. Run: python3 tools/fetch_data.py --id {id} --route {route}
14 3. If comparison route: python3 tools/compare_data.py --cache-dir ...
15 4. Spawn analysis agent (uses .claude/agents/analyzer.md)
16 5. Run: python3 tools/verify_grounding.py --cache-dir ...
17 6. Run: python3 tools/generate_report.py --output reports/{id}.html
18 7. Present summary + report link
19 8. Save outcome to memory
20
21 ## Constraints
22 - Numbers come from files, never from agent reasoning
23 - Single system failure: continue with partial results
24 - Agent confidence LOW: warn user explicitly

```

The frontmatter uses official Claude Code fields: `name` (display name, optional - uses directory name if omitted), `description` (what it does), `when_to_use` (trigger phrases), `allowed-tools` (pre-approved tools), `context`: `fork` (run in subagent), and `agent` (which subagent type). Only the description loads at startup (~100 tokens). The full body loads only when triggered.

Pillar 4: Agent Contracts and Orchestration

What Claude Code Provides (Built-in)

- **Agent definitions.** `.claude/agents/*.md` files with YAML frontmatter. Required fields: `name` and `description`. The markdown body becomes the subagent's system prompt.
- **Agent tool.** Spawns a subagent with its own isolated context window. Returns results to parent. Subagent receives only its system prompt plus environment details - not the parent's full conversation.
- **Tool restriction.** Use `disallowedTools` (denylist) or `tools` (allowlist) to control what a subagent can do. The subagent literally cannot call blocked tools.
- **Model selection.** Each agent can specify `model`: `sonnet`, `opus`, `haiku`, a full model ID, or `inherit` (uses parent's model).

What You Build (Custom Architecture)

- **The contract pattern.** Defining explicit contracts per agent - what it can do, what it cannot do, what it receives, what it returns. This is your architectural discipline on top of the tool restriction feature.
- **Orchestration topology.** Sequential chains, parallel fan-out, conditional routing, escalation patterns. Claude Code gives you the Agent tool; you design how agents compose.
- **Structured output convention.** Claude Code does not enforce output format from subagents. Your agent prompts define the expected JSON structure. This is a convention you establish.

Agent Contract Example

File: `.claude/agents/analyzer.md`

```

1  ---
2  name: analyzer
3  description: Interpret pre-computed comparison results and explain business impact.
4    Delegate when cached data files exist and need human-readable interpretation.
5  model: sonnet
6  disallowedTools: system_a, system_b, system_c, Write, Edit
7  ---
8
9  You are a data analysis agent. You interpret pre-computed results
10 and explain their business meaning.
11
12 ## Your Tools
13 - Read: to read cached data files from .claude/cache/
14 - comparison_engine: to run deterministic field comparisons
15
16 ## You Do NOT Have
17 - Any API tools. You cannot reach external systems.
18 - Write or Edit. You cannot modify files.
19
20 ## Your Job
21 Read the comparison results from disk. Classify each difference
22 as TRUE MISMATCH or FORMAT DIFFERENCE. Return structured JSON:
23 {routing_status, differences: [...], confidence, recommended_action}
24
25 ## Critical Rule
26 You interpret results that already exist on disk.
27 You never re-compare values yourself. You never invent data.
```

The Anti-Hallucination Contract

This is the most important contract pattern for any harness handling real business data. The insight: `disallowedTools` is an official Claude Code feature. Using it to structurally prevent data fabrication is your architectural decision.

Step	What Happens	LLM Involved?
1. Fetch	Python script calls APIs, writes to <code>.claude/cache/{id}/data.json</code>	No
2. Block	Agent definition blocks ALL API tools via <code>disallowedTools</code>	N/A (config)
3. Read	Agent uses Read tool on cached files - returns actual file contents	Yes (reads only)
4. Interpret	Agent explains what the data means, classifies differences	Yes (interprets only)
5. Report	Python script reads files directly for all numbers in final output	No

The capability to fabricate is removed, not just discouraged.

I learned this the hard way: an agent with direct API access fabricated plausible data with zero tool uses. The agent produced fake numbers, fake IDs, and fake "proof" it had called the APIs. Adding rules ("don't hallucinate", "always call tools") did not help. Removing the tool access fixed it permanently.

Pillar 5: Tiered Execution

This is entirely custom architecture. Claude Code does not have a concept of "tiers" or "early exit." You build this in your skill's workflow steps.

Tier	Cost	What Happens	Built On
Tier 0: Memory/Cache	Free, instant	Check: has this pattern been seen before? Is there a cached result on disk? If YES: return known answer, skip all expensive work.	Claude Code's auto-memory retrieval
Tier 1: Lightweight Signal	Cheap, ~300ms	One API call to determine the situation. Decision point: which path? Is deep work needed? If clear: return answer, done.	Single MCP tool call
Tier 2: Deterministic Computation	Moderate, ~2s	Fetch data, compare fields, build timelines. Zero LLM. Writes structured results to disk. If conclusive: explain and return.	Python scripts + MCP computation servers
Tier 3: Subagent Investigation	Expensive, ~10s	Spawn specialized subagent with restricted tools. Agent reads heavy data, reasons through it. Returns structured summary. Heavy data destroyed.	Claude Code's Agent tool (context isolation)

Each tier is an exit point. Most queries resolve at Tier 0 or Tier 1 once your memory bank grows. The harness gets faster over time without code changes.

Pillar 6: Deterministic Computation

This is custom architecture built on top of Claude Code's MCP server support.

What Claude Code Provides (Built-in)

- **MCP server integration.** `.mcp.json` declares servers that auto-start via `uv run`. Claude Code manages their lifecycle.
- **Env var expansion.** `${VAR}` syntax in `.mcp.json` for credential management.

What You Build (Custom Architecture)

- **Computation servers.** MCP servers that wrap pure Python functions - field comparison engines, state machines, lookup tables. Same input always produces same output. No API calls, no randomness.
- **Python tools calling MCP servers.** Scripts that spawn MCP servers via JSON-RPC stdio, call their tools, and write results to disk. This replicates what Claude Code does internally, but deterministically and outside of Claude's context.
- **The "numbers path."** A design principle: identify every flow where numbers go from source system to user-facing output. That entire path must be LLM-free. Python fetches, Python compares, Python renders. The LLM only explains meaning.

```
1 Source API
2   |
3   v
4 fetch_data.py -----> raw.json (on disk)
5           |
6           v
7   compare_data.py -----> comparison_results.json (on disk)
8                   |
9                   v
10  generate_report.py -----> final HTML
11
12 At no point does an LLM touch, transform, or relay a number.
13 The LLM explains what the numbers mean. It never transports them.
```

Pillar 7: Portability and Shareability

What Claude Code Provides (Built-in)

- **Project-level .claude/ directory.** Skills (.claude/skills/), agents (.claude/agents/), rules (.claude/rules/), and settings (.claude/settings.json) all live in version control. Anyone who checks out the repo gets the full harness.
- **.mcp.json.** MCP server configs committed to git. Auto-connect on session start. Dependencies auto-install via uv run with inline PEP 723 deps.
- **Settings scoping.** Project settings (.claude/settings.json) apply to all users. Personal overrides go in .claude/settings.local.json (gitignored) or ~/.claude/settings.json .
- **CLAUDE.local.md.** Personal project-specific preferences (gitignored). For things like your sandbox URLs or local test data - not for sharing the harness.

What You Build (Custom Architecture)

- **Zero-prompt operation.** Pre-approving tool patterns in .claude/settings.json so business users never see permission prompts during normal workflow. This ships with the repo.
- **Credential management.** .env.example (committed, documents what is needed) + .envrc (direnv auto-loads credentials on cd) + \${VAR} expansion in .mcp.json . The harness ships without secrets; each user fills in their own.
- **Self-contained repo.** The entire harness - skills, agents, rules, MCP servers, Python tools, settings - is committed. No external dependencies beyond credentials. Clone and run.

Colleague Setup

```
1 git clone [repo]
2 cp .env.example .env      # fill in your system credentials
3 claude                    # start session
4                             # .claude/skills/ loaded automatically
5                             # .claude/agents/ available immediately
6                             # .claude/rules/ applied to every session
7                             # MCP servers auto-connect via .mcp.json
```

No npm. No pip. No Docker. No deployment. The harness is the repo. If a colleague cannot run it in 60 seconds after cloning, it is not production-grade.

How the Pillars Compose: A Single Execution

All seven pillars working together in one workflow invocation:

User: /investigate ABC-123

Step	Pillar	What Happens	Built-in vs Custom
1	Skills	Skill triggered. Body loads into context.	Built-in: skill lazy-loading

2	Memory	Auto-memory already loaded. Organized as facts/episodes/procedures.	Built-in: MEMORY.md auto-load. Custom: categorization
3	Tiered	Check memory for known pattern. No match. Proceed to Tier 1.	Custom: tiered execution in skill steps
4	Context	Main session at ~7K tokens. Lean.	Custom: token budget discipline
5	Deterministic	Tier 2: fetch + compare. Write results to disk.	Built-in: MCP auto-start. Custom: Python tools, disk cache
6	Agents	Spawn analysis subagent with restricted tools.	Built-in: Agent tool, disallowedTools. Custom: contract pattern
7	Context	Subagent's 20K destroyed. Main still lean.	Built-in: subagent context isolation
8	Deterministic	Verification + report generated from disk.	Custom: grounding verification, LLM-free report
9	Memory	Save investigation outcome.	Built-in: auto-memory write. Custom: promotion workflow
10	Portable	Same result whether run by you or your colleague.	Built-in: .claude/ committed, .mcp.json. Custom: zero-prompt settings, credential pattern

Lessons Learned

- Context engineering is the harness.** Claude Code gives you auto-compaction and subagent isolation. Everything else - token budgets, disk boundaries, structured summaries - is your architecture. Without this discipline, nothing else matters.
- Know what is built-in vs custom.** Conflating Claude Code features with your own patterns confuses your team and creates false expectations. Be explicit: "this is a Claude Code feature" vs "this is our architectural pattern built on top."
- Memory without a lifecycle is a junk drawer.** Auto-memory is a Claude Code feature. The three-bank categorization, the promotion workflow from personal to shared, the retrieval strategy - those are your architecture. Design them deliberately.
- Skills are not prompts.** Claude Code ships the SKILL.md format with lazy loading. But a skill that just says "do the thing" is a prompt, not a workflow. Encode steps, error handling, constraints, and quality gates. The format is free; the workflow design is your work.
- disallowedTools is your strongest tool.** Claude Code lets you structurally block tools from subagents. This is not just a permission - it is an architectural guarantee. Use it to make fabrication impossible, not just discouraged.
- LLMs will shortcut tool calls.** If an agent CAN generate plausible output without calling tools, eventually it will. I observed this firsthand: zero tool uses, fabricated data with fake citations. Remove the capability. Do not just add rules.
- Separate data transport from interpretation.** LLMs explain meaning well. They transport data unreliably. Deterministic scripts fetch, compare, and render. LLMs interpret and explain. This separation is your architecture on top of Claude Code's MCP support.
- Tiered execution is free performance.** Claude Code does not give you tiers. You build them in skill steps. But once built, most queries resolve at Tier 0 (memory) or Tier 1 (lightweight check). The harness gets faster over time as memory grows.
- Portability is the default if you commit .claude/. Claude Code's** project-level structure (.claude/skills/, agents/, rules/, settings.json, .mcp.json) is designed to be committed. The harness IS the repo. The only custom work is the credential pattern (.env.example + direnv) and zero-prompt settings. A harness that only works on your machine is a prototype.

Closing

The primitives are all there in Claude Code. The harness is how you compose them. Start with one pillar - context engineering or agent contracts - and grow from there. Each pillar reinforces the others. The system gets more reliable and faster with every use as memory grows and patterns solidify into shared rules.

TAGS

AGENTIC AI AI Agent Claude Code Context Engineering Mcp

2 Comments



venkatamandavilli

Active Participant

2 weeks ago



1 Like

@Mohan-Sharma

This is a really strong framing of agentic systems. I like the idea that the harness matters more than the raw model, because in production the real challenge is not just intelligence — it is control, context, verification, and workflow design. The seven-pillar approach makes the architecture feel practical and grounded, especially for teams trying to move from experiments to something reliable enough for real work.



SAP

Mohan-Sharma

Product and Topic Expert

2 weeks ago

 0 Likes

[@venkatamandavilli](#) Exactly - and the key insight is that the harness pattern is orchestrator-agnostic. Whether you build on Claude Code, Pi, or LangChain's DeepAgents, the same seven concerns show up. The orchestrator choice changes the implementation details, but the architectural problems you have to solve stay the same.

You must be a registered user to add a comment. If you've already registered, sign in. Otherwise, register and sign in.

[Comment](#)



Top Liked Authors

- 

SAP

Swathy_S

32
- 

SAP

Dimitri

25
- 

SAP

Mohan-Sharma

24
- 

venkatamandavilli

18
- 

SAP

yoramfriedman

16
- 

L

SAP ladenaw

12
- 

A

SAP Abhishek_Kumar11

10
- 

D

SAP DEEPA_DORAIRAJ

10
- 

MrRaj

7
- 

C

CasperCui

7

[View all](#) >